

Neural Networks

Sangmyung Ha

May 10, 2024

Outline

Neural Network

Optimization

Regularization

Extensions

Tutorials

References

- ▶ Trevor Hastie, Robert Tibshirani, Jerome Friedman (2009), "The Elements of Statistical Learning (ESL)", Chapter 11
- ▶ Christopher M. Bishop (2006), "Pattern Recognition and Machine Learning", Chapter 5
- ▶ Ian Goodfellow, Yoshua Bengio, Aaron Courville (2016), "Deep Learning", Chapter 6, Chapter 7
- ▶ Srivastava et al. (2014) "Dropout: A Simple Way to Prevent Neural Networks from Overfitting ", Journal of Machine Learning Research
- ▶ MIT Course slides (6S.191), <http://introtodeeplearning.com>

Perceptron

Perceptrons are the building blocks of neural networks. It consists of passing a linear combination of inputs to a nonlinear function.

$$y = g \left(w_0 + \sum_{j=1}^m w_j x_j \right)$$

$w_j, j = 1, \dots, m$ are called *weights*, w_0 is called a *bias*, and the nonlinear map g is called *activation function*. Commonly used activation functions include:

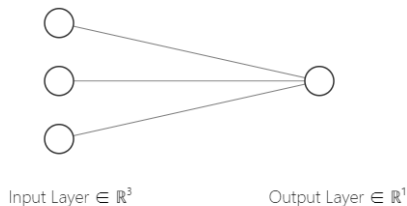
- ▶ Logit: $g(z) = 1/(1 + e^{-z})$
- ▶ Hyperbolic Tangent: $g(z) = (e^z - e^{-z})/(e^z + e^{-z})$
- ▶ Rectified Linear Unit (ReLU): $g(z) = \max(0, z)$

Perceptron as a vector-vector multiplication

Let $x = (1, x_1, \dots, x_m)'$ and $w = (w_0, w_1, \dots, w_m)'$ be $m + 1$ dimensional vectors collecting one observation and the coefficients. We may write

$$y = g \left(w_0 + \sum_{j=1}^m w_j x_j \right)$$
$$\implies y = g(w'x)$$

Graphical depiction of a perceptron:



A stack of perceptrons: matrix-vector multiplication

One perceptron takes as input $x \in \mathbb{R}^{m+1}$ and gives as output $y \in \mathbb{R}$. Suppose we have a stack of perceptrons giving as output $y \in \mathbb{R}^L$.

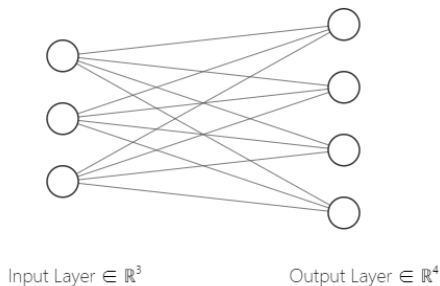
$$y_l = g \left(w_{0,l} + \sum_{j=1}^m w_{j,l} x_j \right)$$

for $l = 1, \dots, L$. Let $w^l = (w_{0,l}, w_{1,l}, \dots, w_{m,l})'$, and $W = (w^1, \dots, w^L)'$. We may write

$$\begin{aligned} y_l &= g \left(w_{0,l} + \sum_{j=1}^m w_{j,l} x_j \right) \\ \implies y &= g(Wx) \end{aligned}$$

where $y = (y_1, \dots, y_L)' \in \mathbb{R}^L$ and g is applied to each element of Wx in the second line.

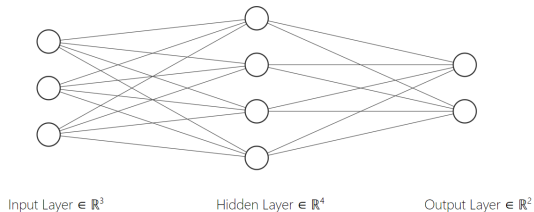
A stack of perceptrons



Given $x \in \mathbb{R}^{(m+1)}$, left multiplying by a matrix of weights $W \in \mathbb{R}^{L \times (m+1)}$ gives an output $z = Wx \in \mathbb{R}^L$. We apply g to each element of z to get $y = g(Wx) \in \mathbb{R}^L$. Note that each row of W collects coefficients for each node in the output layer:

$$W = \begin{pmatrix} w^{1'} \\ \vdots \\ w^{L'} \end{pmatrix}$$

Neural Network



Given $x \in \mathbb{R}^{(m+1)}$, left multiply by $W^{(1)} \in \mathbb{R}^{L \times (m+1)}$ and map the outcome $z^{(1)} = W^{(1)}x$ through g to get $y^{(1)} = g(z^{(1)}) \in \mathbb{R}^L$. Given $(1, y^{(1)'})' \in \mathbb{R}^{(L+1)}$, left multiply by $W^{(2)} \in \mathbb{R}^{K \times (L+1)}$ and map the output $z^{(2)} = \begin{matrix} W^{(2)} \\ K \times (L+1) \end{matrix} \begin{matrix} (1, y^{(1)'})' \\ (L+1) \times 1 \end{matrix}$ through σ to get the final

output $y^{(2)} = \sigma(z^{(2)})$. In summary,

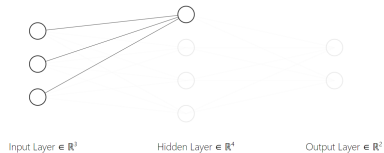
$$y^{(2)} = \sigma(W^{(2)}(1, g(W^{(1)}x)'))' \in \mathbb{R}^K$$

For notational simplicity, we'll write

$$y^{(2)} = \sigma(W^{(2)}g(W^{(1)}x))) \in \mathbb{R}^K$$

and understand that a bias term will always appears in each layer.

Neural Network



Functions used in hidden layers are called **activation function**. Typically, logit, hyperbolic tangent or ReLu is used.



Functions used in output layers are called **output function**. For regression, the identity function is used. For classification, softmax function ($\sigma_k(z) = e^{z_k} / \sum_{l=1}^K e^{z_l}$) is used.

Neural Network

The nonlinear mapping allows the span of the output variable to be much larger than the span of X . Let $X \in \mathbb{R}^{N \times (m+1)}$ be the matrix of observations. Without nonlinear maps, the evaluation of the neural net above amounts to

$$Y_{N \times K} = \begin{pmatrix} 1_{N \times 1}, & XW'_1 \end{pmatrix}_{N \times (1+L)} \begin{pmatrix} W'_2 \end{pmatrix}_{(1+L) \times K} \in \mathbb{R}^{N \times K}$$

where $(1_{N \times 1}, XW'_1) \in \mathbb{R}^{N \times (L+1)}$. With nonlinear mapping,

$$Y = \sigma(g(XW'_1)W'_2)$$

where g and σ have been applied to each element.

Deep neural network

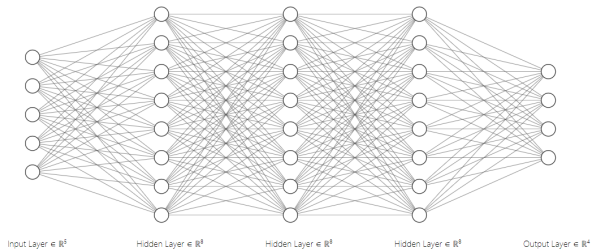


Figure 1: NN with 3 hidden layers, each with 8 nodes

Neural networks with multiple hidden layers are called deep neural networks. In this case, controlling the complexity of a neural network in order to fight overfitting becomes crucial.

Neural Network

Neural network models introduced above are sometimes called **feed forward** neural networks. The name comes from the fact that data flows only in the forward direction, i.e., there are no feedback connections in which outputs of the model are fed back into itself. Similarly, evaluating the output value of a neural network is called **forward propagation**.

A neural network layer is called **dense** if all inputs are densely connected to all outputs in that layer. A neural network model can be modified to have sparse connections between layers or have skip-layer connections. One of the advantages of neural network is that people can use prior domain knowledge to build architecture of neural network.

A neural network without hidden layers and a sigmoid output function is equivalent to logistic regression. A neural network using linear activation and output function is equivalent to multivariate regression.

Optimization: Loss function

Neural networks are trained by minimizing a loss function. For regression, sum-of-squared errors are used:

$$\begin{aligned} L(W) &= \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (y_{ik} - f_k(x_i))^2 \\ &= \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))' (y_i - f(x_i)) \end{aligned}$$

For classification, cross-entropy errors are used.

$$\begin{aligned} L(W) &= -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log f_k(x_i) \\ &= -\frac{1}{N} \sum_{i=1}^N y_i' \log f(x_i) \end{aligned}$$

where W collects the parameters of the neural network.

Optimization: Gradient Descent

Given the unconstrained optimization problem below,

$$\min_{W \in \mathbb{R}^p} L(W)$$

the **gradient descent** algorithm is as follows:

Gradient Descent

1. **Given** a starting point $W \in \mathbb{R}^p$
 2. **Repeat**
 - (a) Evaluate the steepest descent direction at W : $\Delta W = -\nabla L(W)$.
 - (b) Line Search: Choose step size t via exact (or backtracking) line search
 - (c) Update: $W \leftarrow W + t\Delta W$
 3. **until** stopping criterion is satisfied.
-

Note that the gradient descent algorithm does not guarantee convergence to the global optimum since the loss function is not a convex function of parameters.

Optimization: Gradient Descent

In **exact line search**, t is chosen to minimize f along the ray $\{x + t\Delta x | t \geq 0\}$:

$$t = \arg \min_{s \geq 0} f(x + s\Delta x)$$

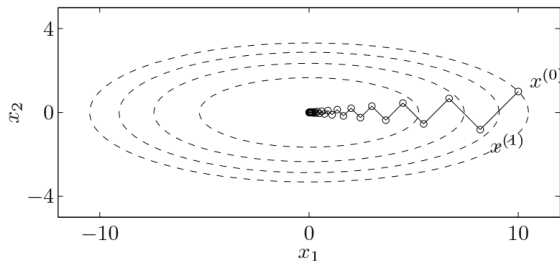
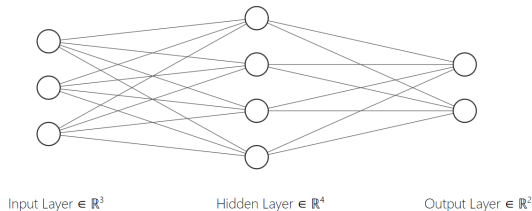


Figure 9.2 Some contour lines of the function $f(x) = (1/2)(x_1^2 + 10x_2^2)$. The condition number of the sublevel sets, which are ellipsoids, is exactly 10. The figure shows the iterates of the gradient method with exact line search, started at $x^{(0)} = (10, 1)$.

Gradient Descent: Gradient computation

The process of computing gradient is called **back propagation**. Redundant computation can be eliminated by the use of the chain rule. Consider one hidden layer regression model with m inputs, L hidden nodes, and K outputs.



Consider regression problem:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W), \quad L_i(W) = \sum_{k=1}^K (y_{ik} - f_k(x_i))^2$$

Back Propagation: Matrix Calculus

Let $z_i^{(1)} = W^{(1)}x_i$, $z_i^{(2)} = W^{(2)}g(z_i^{(1)})$, $y_i^{(1)} = g(z_i^{(1)})$, and $y_i^{(2)} = \sigma(z_i^{(2)})$. The function value of the neural network is given by $y_i^{(2)} = \sigma(W^{(2)}g(W^{(1)}x_i)) \in \mathbb{R}^K$. Let $W_l^{(2)'}$, $W_l^{(1)'}$ be the l -th row of $W^{(2)}$, and $W^{(1)}$. Then, the chain rule gives us:

$$\frac{\partial L_i(W)}{\partial W_l^{(2)'}} = \frac{\partial L_i(W)}{\partial y_i^{(2)'}} \frac{\partial y_i^{(2)}}{\partial z_i^{(2)'}} \frac{\partial z_i^{(2)}}{\partial W_l^{(2)'}}$$
$$\frac{\partial L_i(W)}{\partial W_l^{(1)'}} = \frac{\partial L_i(W)}{\partial y_i^{(2)'}} \frac{\partial y_i^{(2)}}{\partial z_i^{(2)'}} \frac{\partial z_i^{(2)}}{\partial y_i^{(1)'}} \frac{\partial y_i^{(1)}}{\partial z_i^{(1)'}} \frac{\partial z_i^{(1)}}{\partial W_l^{(1)'}}$$

where $\frac{\partial L_i(W)}{\partial y_i^{(2)'}} = -2(y_i - y_i^{(2)})'$, $\frac{\partial y_i^{(2)}}{\partial z_i^{(2)'}} = \frac{\partial \sigma(z_i^{(2)})}{\partial z_i^{(2)'}}$, $\frac{\partial z_i^{(2)}}{\partial y_i^{(1)'}} = W^{(2)}[:, 2 : L + 1]$,

$\frac{\partial z_i^{(1)}}{\partial W_l^{(1)'}} = (0_{(m+1) \times (l-1)}, x_i, 0_{(m+1) \times (L-l-1)})'$, and

$\frac{\partial z_i^{(2)}}{\partial W_l^{(2)'}} = (0_{(L+1) \times (K-1)}, y_i^{(1)}, 0_{(L+1) \times (K-l-1)})'$.

Optimization: Stochastic Gradient Descent

The loss function is the average of the loss from each observation:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W), \quad L_i(W) = \sum_{k=1}^K (y_{ik} - f_k(x_i))^2$$

Thus, gradient descent would repeat:

$$\begin{aligned} W^{(k)} &= W^{(k-1)} - t_k \nabla L(W^{(k-1)}) \\ &= W^{(k-1)} - t_k \frac{1}{N} \sum_{i=1}^N \nabla L_i(W^{(k-1)}) \quad k = 1, 2, 3, \dots \end{aligned}$$

where t_k is found by line search. However, computing loss from every observation at each step can be computationally expensive.

Optimization: Stochastic Gradient Descent

In comparison, **stochastic gradient descent** or SGD repeats:

$$W^{(k)} = W^{(k-1)} - t_k \nabla L_{i_k}(W^{(k-1)}) \quad k = 1, 2, 3, \dots$$

where $i_k \in \{1, \dots, N\}$ is some chosen index at iteration k .

Note that

$$\mathbb{E}[\nabla L_{i_k}(W)] = \nabla L(W)$$

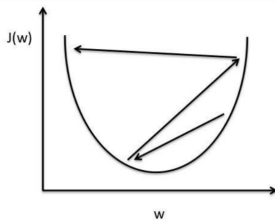
so we can view SGD as using an unbiased estimate of the gradient at each step.

Stochastic gradient descent is implemented by randomly selecting a sample without replacement at each iteration. A complete pass through the whole sample is called one **epoch**, e.g., computing 5 epochs requires $5N$ iterations.

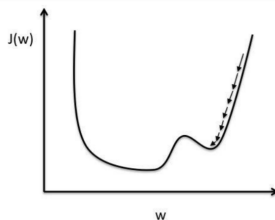
Stochastic Gradient Descent

Unlike in gradient descent, stochastic gradient descent commonly uses fixed step size, i.e., $t_k = t$, $k = 1, 2, 3, \dots$

In this case, choosing too small a step size can result in getting stuck in local minimum. Also, choosing too large a step size can result in overshooting and cause the objective value to diverge.



Large learning rate: Overshooting.



Small learning rate: Many iterations until convergence and trapping in local minima.

In neural networks, step size is commonly referred to as **learning rate**.

Stochastic Gradient Descent

Instead of using fixed step sizes, many adaptive step size algorithms have been proposed in context of neural networks:

- ▶ Adam
- ▶ Adagrad
- ▶ Adadelta
- ▶ RMSProp

Stochastic Gradient Descent with mini-batch

Since stochastic gradient direction can be an unreliable estimate for the true gradient, **mini-batch** stochastic gradient descent is often used. Here, we choose a random subset $I_k \subset \{1, \dots, N\}$, of size b and repeat:

$$W^{(k)} = W^{(k-1)} - t_k \frac{1}{b} \sum_{i \in I_k} \nabla L_{i_k}(W^{(k-1)}) \quad k = 1, 2, 3, \dots$$

As with before, samples are drawn without replacement for each epoch. Suppose $N = 950$ and $b = 100$, one epoch requires 10 iteration, with the last batch size comprising 50 samples.

Regularization: Dropout

In **dropout**, a portion of units in each layer is set to 0 during training for each mini-batch. The probability of a unit being included is a hyperparameter fixed before training begins. Typically, an input unit is included with a probability of 0.8, and a hidden unit is included with a probability of 0.5.

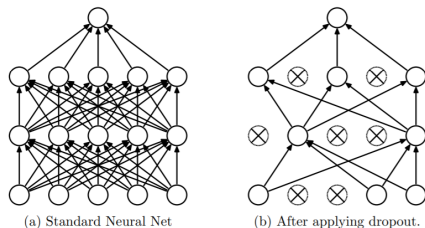


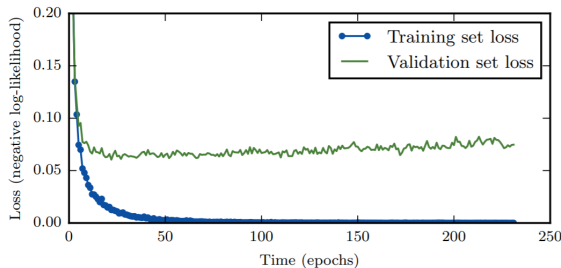
Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout forces the network to not rely on any one node and prevents the network from memorizing the training set.

Regularization: Early stopping

In **early stopping**, a portion of the training sample is set aside to be used to compute validation error. Training stops once the validation error increases for a fixed number of epochs.

Early stopping is a method to stop training before the network has a chance to overfit. Because starting values for weights are chosen to be random values near zero, early stopping has the effect of regularizing the size of weights.



Regularization: Norm penalties

L_2 and L_1 penalties can be used to regularize the weights. L_1 penalty results in sparse solution. Typically, only the weights, excluding the biases, are penalized.

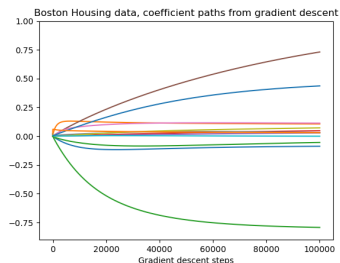
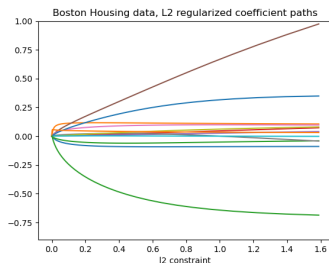
$$\tilde{L}(W) = L(W) + P_\lambda(W)$$

$$P_\lambda(W) = \begin{cases} \lambda \|W\|_1 & \text{for } L_1 \text{ penalty} \\ \lambda \|W\|_2^2 & \text{for } L_2 \text{ penalty} \end{cases}$$

Here, λ is a tuning parameter which controls the amount of regularization.

Regularization

Relationship between early stopping and L_2 norm penalization:



Data: Boston Housing data

Left is the coefficient path derived from ridge regression, plotted against L_2 norm constraint size. Right is the coefficient path from computing OLS solution with gradient descent. The coefficient here is plotted against gradient descent steps.

Recurrent Neural Network

Recall that passing through one layer involves one weight matrix $W \in \mathbb{R}^{L \times (m+1)}$, where m is the input dimension and L is the number of nodes, and one nonlinear function g . Evaluation of one sample x_t to produce one prediction $\hat{y}_t \in \mathbb{R}^K$ can be written as

$$\hat{y}_t = f(x_t)$$

where $f : \mathbb{R}^m \rightarrow \mathbb{R}^K$ is a function consisting of various matrix multiplication and nonlinear mappings.

Recurrent Neural Network

A recurrent neural network is a modification to a feed-forward neural network that exploits the time series structure of input data. Recurrent neural network consists of two neural networks

$$h_t = f^{(1)}(x_t, h_{t-1})$$

$$\hat{y}_t = f^{(2)}(x_t, h_t)$$

where $f^{(1)} : \mathbb{R}^{m+p} \rightarrow \mathbb{R}^p$ specifies how the latent state $h_t \in \mathbb{R}^p$ evolves, and $f^{(2)} : \mathbb{R}^{m+p} \rightarrow \mathbb{R}^K$ specifies how predictions are made given today's observation and latent state.

Recurrent Neural Network

Example: Suppose we do not model any hidden layers in both $f^{(1)}, f^{(2)}$. Then, we have

$$h_t = g(W_{xh}x_t + W_{hh}h_{t-1})$$

$$\hat{y}_t = g(W_{xy}x_t + W_{hy}h_t)$$

Suppose further that we use identity map as the output function in $f^{(2)}$ and that $W_{xy} = 0$. Then, we have

$$h_t = g(W_{xh}x_t + W_{hh}h_{t-1})$$

$$\hat{y}_t = W_{hy}h_t$$

Recurrent Neural Network

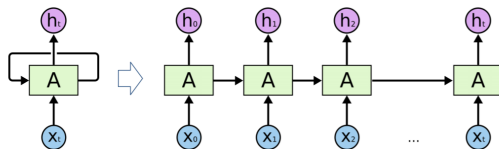
The loss function is given in the same way as in feedforward neural network. I.e., in the regression case

$$L(W) = \frac{1}{T} \sum_{t=1}^T L_t(W) = \frac{1}{T} \sum_{k=1}^K (y_{kt} - f_k(x_t))^2$$

Estimation is done with gradient descent, where the gradient is computed with backpropagation through time (chain rule). In calculating $\nabla L_t(W)$, we need to calculate and store the gradients $\nabla L_s(W)$, $s > t$ first.

Recurrent Neural Network

One way to understand RNNs is to expand the input space $\mathbb{R}^m$ to $\mathbb{R}^{mT}$ but to restrict the model by using "parameter sharing".



"parameter sharing" $\iff$ modeling "stationarity" in time

Convolutional Neural Network

Suppose we have a pictorial data, where each sample X_{ij} has two indices. Also suppose that the data X_{ij} satisfies (covariance) 'spatial' stationarity. In other words, X_{ij} is (covariance) stationary in each direction so that for any i_1, j_1, i_2, j_2 and for any h_1, h_2 , we have

$$\begin{aligned} & \mathbb{E}[(X_{i_1, j_1} - \mathbb{E}X_{i_1, j_1})(X_{i_2, j_2} - \mathbb{E}X_{i_2, j_2})] \\ &= \mathbb{E}[(X_{i_1+h_1, j_1+h_2} - \mathbb{E}X_{i_1+h_1, j_1+h_2})(X_{i_2+h_1, j_2+h_2} - \mathbb{E}X_{i_2+h_1, j_2+h_2})] \end{aligned}$$

		$X_{0,1}$		
$X_{-2,0}$			$X_{1,0}$	
	$X_{-1,-1}$			

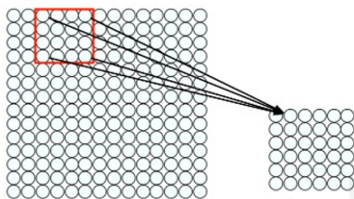
$$\mathbb{E}[(X_{-2,0} - \mathbb{E}X_{-2,0})(X_{-1,-1} - \mathbb{E}X_{-1,-1})] = \mathbb{E}[(X_{0,1} - \mathbb{E}X_{0,1})(X_{1,0} - \mathbb{E}X_{1,0})]$$

Convolutional Neural Network

How to exploit this structure? $\implies$

1. Do not flatten the 'matrix' valued data.
2. Instead of matrix multiplication of the input vector and a matrix of weights, introduce a notion of 'convolution' of input matrix and a matrix of weights.
3. If input is given by a 3D rectangular cube (cuboid), use 'convolution' of input cuboid and a cuboid of weights.

Feature Extraction with Convolution



- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

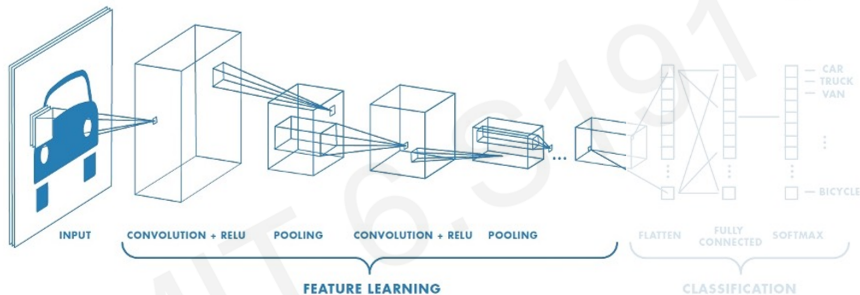
1) Apply a set of weights – a filter – to extract **local features**

2) Use **multiple filters** to extract different features

3) **Spatially share** parameters of each filter

Figure 3: Slide 25 from MIT deeplearning L3

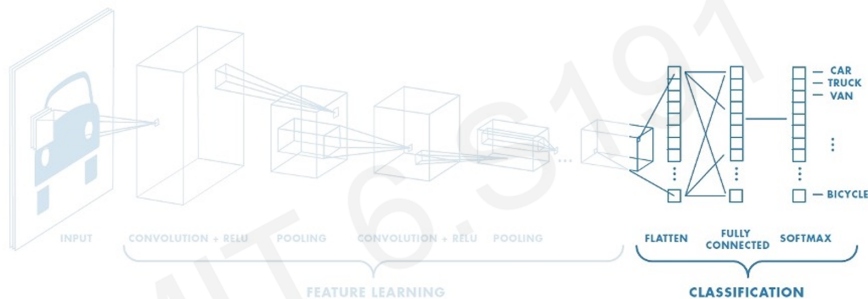
CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

Figure 4: Slide 51 from MIT deeplearning L3

CNNs for Classification: Class Probabilities



- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Figure 5: Slide 52 from MIT deeplearning L3

Convolutional Neural Network

A convolutional neural network is a modification to a fully connected neural network that uses the spatial structure of input data. Generally, the input of CNN consists of $N \times M \times 3$ dimensional rectangular box. As with RNN, think of this as $3NM$ number of inputs, but use parameter sharing to reduce the number of parameters.

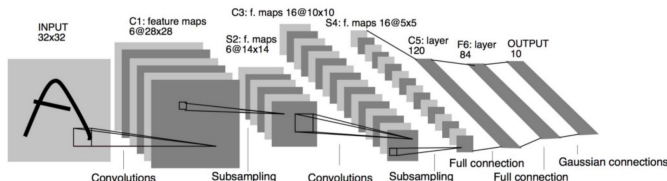


Figure 6: Architecture of LeNet5 (LeCun et al, 1998)

Tutorials on implementation

- ▶ MIT Lab session in 6S.191:
<https://github.com/aamini/introtodeeplearning>
- ▶ Tensorflow Tutorials:
<https://www.tensorflow.org/tutorials>